

Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis

Abdullah Faiyaz

Department of CSE, BUET

2005065@ugrad.cse.buet.ac.bd

Jarin Tasnim

Department of CSE, BUET

2005083@ugrad.cse.buet.ac.bd

Sadnam Faiyaz

Department of CSE, BUET

2005111@ugrad.cse.buet.ac.bd

Md. Mostofa Akbar

Department of CSE, BUET

mostofa@cse.buet.ac.bd

Abstract—Detection of malware on Android has come a long way, with both static and dynamic analysis methods. But the question of implementing them in real-life, constrained environments with limited battery, CPU, and memory still remains answered by very few approaches. Taking the question seriously, we present *VigiDroid*, a fully offline static analysis framework that runs entirely on the device itself. We have experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and raw APK byte sequences), pairing each with lightweight classifiers such as XGBoost, multilayer perceptrons, and 1D CNNs. Every model is exported to ONNX and executed on-device via ONNX Runtime, enabling direct evaluation on Android devices. In addition to accuracy and F1-score, we profile feature extraction, vectorization, and inference times, CPU utilization, and memory consumption. Our results suggest that lightweight static features extracted from DEX headers, permissions, intents, and broadcast receivers offer one of the most favorable accuracy-resource trade-offs (96.99% accuracy and F1 of 0.9503 with only 0.9 ms CPUtime and 0.5 ms walltime).

Index Terms—Android malware detection, on-device inference, static analysis, multi-DEX, XGBoost, 1D-CNN, ONNX Runtime

I. INTRODUCTION

Over the years, researchers have made real progress in detecting Android malware — static analysis, dynamic analysis, and hybrid approaches have all shown impressive results on paper. But most of these techniques were designed and evaluated on powerful machines, not on the phones people actually carry around with limited battery, CPU, and memory. In this paper, we introduce *VigiDroid*, a fully offline static analysis framework that runs entirely on the device itself. We experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and even raw APK byte sequences) paired with lightweight models such as XGBoost, multilayer perceptrons, and 1D CNNs. All models are deployed using ONNX Runtime and evaluated directly on Android devices. In addition to detection accuracy and F1-score, we profile feature extraction, vectorization and inference time, CPU utilization, and memory consumption. Preliminary findings suggest that lightweight static features derived from DEX headers, permissions, intents, and receivers provide an attractive balance between detection effectiveness and resource efficiency, making them particularly suitable for on-device malware detection.

II. BACKGROUND & RELATED WORKS

Android malware detection has been studied extensively using both static and dynamic analysis. Like permission-based

linear models [2], broadcast-receiver mining [4], permission extraction frameworks (MLDP) [8], byte-level 1-D CNNs on raw APK tails [1], and structural DEX/manifest fusion architectures by MSFDroid [11]. Early efforts like SeqMobile [5] showed that on-device detection is possible by using RNN-based models on permissions, intents, and API call sequences. SAMADroid [6] combined lightweight on-device checks with cloud-based dynamic analysis. However, its reliance on certain system privileges has become problematic as modern Android versions have tightened security restrictions. DL-Droid [7] ran dynamic analysis directly on real devices, but it required installing and fully executing each app which make it impractical for everyday scanning. Overall, these works highlight that building a malware detector that is both accurate and lightweight enough to run comfortably on a real phone remains an unsolved problem.

III. PROPOSED METHODOLOGY

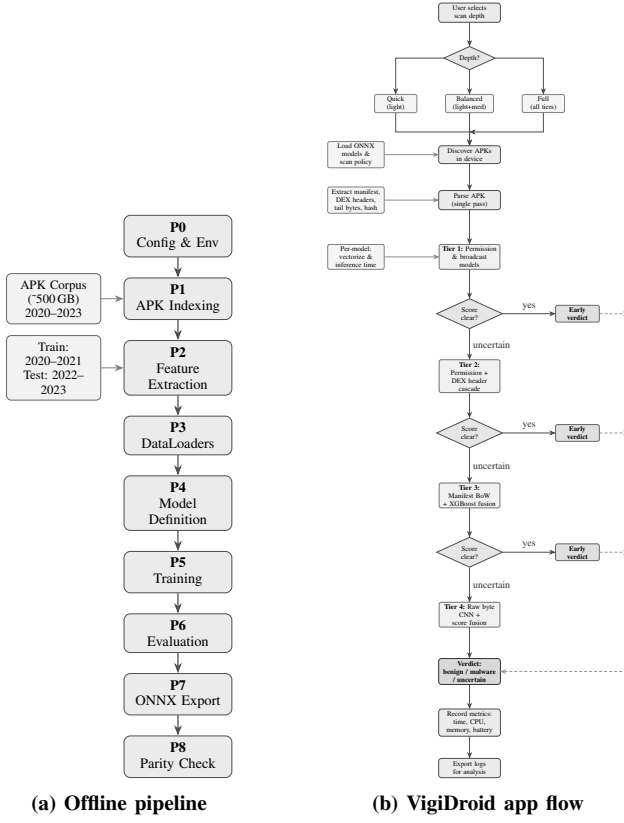


Fig. 1. Proposed VigiDroid methodology. *Left*: the offline model-development pipeline, with export and model-family details omitted for compactness. *Right*: the on-device scan pipeline that selects a scan depth, extracts features locally, runs ONNX inference, fuses scores, and exports resource metrics.

IV. EXPERIMENTS

We evaluate each model in terms of detection performance (Accuracy, F1-score, ROC-AUC on a temporal holdout set from 2022-2023) and on-device feasibility (CPU time, memory usage, battery consumption, and inference latency measured on a physical Android device). Table I summarizes the results obtained using VigiDroid under full-scan mode. Among all configurations, Dex+Broadcast Fusion achieved the best overall performance with 96.99% accuracy and 0.9503 F1-score while requiring less than 1 ms inference time and negligible memory overhead. In contrast, API-based models such as XGBoost incurred substantially higher processing costs due to feature extraction overhead despite competitive detection performance. While prior approaches such as SeqMobile and DroidCAT reported higher detection accuracy, their reliance on sequence processing or behavioral analysis introduces significant computational overhead, limiting their practicality for real-time on-device deployment.

inferenceTime_vs_apkSize.png

Fig. 2. Inference time vs APK size. Though inference time stays flat across APK sizes for all models, the cost is driven almost entirely by the feature domain, not the APK size itself.

sample_model_vs_accuracy_resources.png

Fig. 3. This figure visualises trade-off directly: DEX-header-based models cluster in the high-accuracy, low-resource corner of the plot, while XGBoost sits isolated at high resource cost.

At the top end, Dex+Broadcast Fusion achieves the highest accuracy (96.99%) and F1 (0.9503) among all models, closely followed by MLP-H and the Dex+Manifest Dual model, both reaching 96.86% accuracy. XGBoost posts the strongest ROC-AUC at 0.9913, reflecting its ability to rank samples well across thresholds, though its flat accuracy and F1 are more modest (93.72% and 0.8884 respectively). Permission-only models like LinRegDroid and MLDP-pruned sit at the lower end of the accuracy range (91–93%), confirming that richer structural features yield better discrimination. The 1D-CNN similarly draws more resources than its accuracy warrants (1.7ms CPU, 8.4 battery units) for only 90.05% accuracy. In contrast, the DEX-header family proves highly efficient: MLP-H, MLDP+Dex Cascade, and Dex+Broadcast Fusion all

TABLE I

EXPERIMENTAL COMPARISON OF ON-DEVICE MALWARE DETECTION MODELS. OFFLINE METRICS ARE COMPUTED ON THE COMPUTER TEMPORAL TEST SPLIT; RESOURCE COLUMNS ARE DERIVED FROM ON-DEVICE `ALL_SCAN_METRICS.JSON`.

Model	Features	Acc.	F1	ROC-AUC	CPU	Mem.	Batt.	Time	Feasible
XGBoost	Manifest perms, intents, APIs	0.9372	0.8884	0.9913	97.4 ms	0.02 MB	477.102	100.0 ms	low
1D-CNN	Last 1024 APK bytes	0.9005	0.8486	0.9592	1.7 ms	0.01 MB	8.433	1.7 ms	low
MLP-H	Dex header, sum-pooled	0.9686	0.9478	0.9675	0.7 ms	0.01 MB	1.997	0.4 ms	high
Dex+Manifest ASCNN	Dex header + manifest BoW	0.9411	0.9064	0.9685	7.8 ms	0.03 MB	36.045	7.9 ms	medium
Dex+Manifest Dual	Dex header + manifest, dual-branch	0.9686	0.9483	0.9805	7.9 ms	0.02 MB	36.84	7.9 ms	medium
LinRegDroid	Manifest permission flags	0.9175	0.8645	0.9259	1.0 ms	0.00 MB	3.008	0.7 ms	low
MLDP-pruned	MLDP-selected permissions	0.9319	0.8855	0.9017	0.6 ms	0.00 MB	1.61	0.4 ms	medium
Broadcast+MLDP	MLDP perms + receiver actions	0.9359	0.8928	0.9373	1.1 ms	0.00 MB	1.76	0.4 ms	medium
MLDP+Dex Cascade	MLDP perms + dex header	0.9581	0.9298	0.9469	0.8 ms	0.01 MB	4.919	0.4 ms	high
Dex+Broadcast Fusion	Dex header + receiver actions	0.9699	0.9503	0.9794	0.9 ms	0.01 MB	2.362	0.5 ms	high
Droidcat	0.9739	0.9739							

complete inference in 0.4–0.5 ms with under 5 battery units consumed. These three are rated high feasibility in Table I.

V. CONCLUSION

We described VigiDroid, a thesis framework for practical on-device Android malware pre-screening that unifies literature-based static detectors through a disciplined train-export-deploy pipeline and tiered ONNX orchestration on Android. The results show that DEX-header-based models offer the best of both worlds, with Dex+Broadcast Fusion reaching 96.99% accuracy and an F1 of 0.9503 while consuming under 1 ms of CPU and 2.4 battery units per scan. At the other extreme, XGBoost’s ROC-AUC of 0.9913 comes at the cost of 97.4 ms CPU and 477 battery units. Across all models, the feature extraction pipeline proved to be the dominant cost factor, not the classifier architecture itself.

ACKNOWLEDGMENT

The authors thank their supervisor and collaborators at the Department of Computer Science and Engineering, BUET, for guidance throughout this thesis.

REFERENCES

- [1] C. Hasegawa and H. Iyatomi, “One-dimensional convolutional neural networks for Android malware detection,” in *Proc. ICCE*, 2020.
- [2] A. Narayanan et al., “LinRegDroid: Detection of Android malware using multiple linear regression,” *Computers & Security*, 2018.
- [3] Tao Peng et al., “A Lightweight Multi-Source Fast Android Malware Detection Model,” *Appl. Sci.*, 2022.
- [4] M. Mohsen and A. Ismail, “Detecting Android malwares by mining statically registered broadcast receivers,” *Int. J. Advanced Computer Science and Applications*, 2017.
- [5] Ruitao Feng et al., “SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices” *25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, 2020.
- [6] Saba Arshad et al., “SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System” *IEEE Access*, 2018.
- [7] Mohammed K. Alzaylaee et al., “DL-Droid: Deep learning based android malware detection using real devices” *IEEE Access*, 2020.
- [8] S. Wang et al., “Permission extraction framework for Android malware detection,” *IEEE Access*, 2019.
- [9] Y. Li et al., “MSFDroid: Multi-stage fusion for Android malware detection,” *IEEE Trans. Information Forensics and Security*, 2022.
- [10] Bragança, H., Kreutz, D., Rocha, V. et al., “MH-1M: A 1.34 Million-Sample Multi-Feature Android Malware Dataset with Rich Metadata,” *Sci Data* 13, 153. <https://doi.org/10.1038/s41597-025-06469-5>, 2026.
- [11] Tao Peng et al., “A Lightweight Multi-Source Fast Android Malware Detection Model,” *Appl. Sci.*, 2022.

- [12] D. Arp et al., “Drebin: Effective and explainable detection of Android malware in your pocket,” in *Proc. NDSS*, 2014.
- [13] ONNX Runtime documentation, <https://onnxruntime.ai/>, accessed 2026.